WOODEX INTERIOR - DIGITAL OPERATIONS

Master Plan

Master Dashboard - Live Page Builder
CRM - Quotations & Invoices - Go-Live

Full-stack implementation plan based on the completed deep audit,
the approved design mockup and the five locked decisions.
87-page website - No-code block builder - Lead-gen CRM with live chat & WhatsApp - Server-PDF quotation pipeline - Vercel hosting.

Status	FOR APPROVAL - development starts after sign-off
Version	1.0 - 28 September 2026
Scope	Phases 0-6 - 13 weeks
Hosting	Vercel (site + admin + API) - Hostinger = domain only
Based on	AUDIT-REPORT-2026-09-28.md + design mockup + admin screenshots

Woodex Admin v2.5 Pro - Master Plan

Target version: Admin **v2.5 Pro** - the React + TypeScript rebuild of the Woodex master dashboard **Strategy:** rewrite the front end - version the back end - preserve the data **Based on:** docs/PROJECT-ANALYSIS-REPORT-2026-10-13.pdf (26 pages, 13 branches, 365 API actions measured) **Prototype:** admin-v25/ - 5 working screens, all 65 routes registered, 70 render checks passing **Date:** 13 October 2026 - **Branch:** arena/4150341e-marketingwoodex

1. Executive summary

Woodex already has the hard part: 365 working server actions, a 20-table database holding real leads, quotations and invoices, 150 public pages, and integrations with WhatsApp, Telegram, Google and an AI assistant. What it does not have is an interface architecture that can carry the next five years, and it has never had a clean production boundary.

This plan keeps everything that works and replaces what does not:

Layer	Decision	Why
Front end	Rewrite in React 18 + TypeScript + Vite	The current admin is 60 IIFE modules, 10,502 lines, sharing 56 mutable VIEWS keys and loading in script order. Three of the four shipped admin copies contain duplicate fragments that break parsing. Nothing about that is repairable in place.
Back end	Version, don't rewrite - api/v2/ in PHP 8.2 over the same MySQL	The 365 actions encode years of business rules that already work. Rewriting them buys nothing and risks everything. Instead: one front controller, a typed envelope, safe persistence, and a capability table.
Data	Preserve - same MySQL, migrate the JSON side-stores in with SQL	wx_leads (32 columns, 4 indexes) is real business data. It is extended, not replaced.
Design	One token contract, two first-class themes ("Woodex Ink")	Ends the v2.1 defect where the sidebar was pinned dark with !important and 21 modules carried 343 literal colours, so the obsidian theme only half-applied.
Delivery	Incremental - v2.1 stays live while v2.5 is built screen by screen	A working studio system must never go dark. Cut over one route at a time behind a flag.

What this delivers: a master dashboard that opens on one honest screen - pipeline, revenue, what needs attention today - and reaches all 57 screens (plus 4 new v2.5 modules) through a typed registry, on a back end where money is calculated on the server, JSON writes cannot corrupt state, and 8 Critical + 28 High open findings are closed rather than carried forward.

What it does not do: it does not put a Node server in production. Hostinger stays Apache + PHP + MySQL. The React build is static files. That keeps hosting cost at zero and risk low.

1.1 Headline targets

Metric	Today (v2.1 Pro)	v2.5 Pro target
Admin modules	64 JS files, no build step	~90 TS modules, one build, content-hashed assets
Bundle	67 unversioned <script> tags, ~1.2 MB vendor	app ~ 80 kB gz + vendor chunks, immutable cache
View registry	62 keys, 20 registered by 2+ modules	1 route = 1 component, typed
Literal colours	343 across 21 files (being swept)	0 - enforced by lint

Metric	Today (v2.1 Pro)	v2.5 Pro target
Auth token	HMAC bearer in sessionStorage (XSS-readable)	HttpOnly SameSite=Lax cookie + CSRF
Money maths	recalculated in the browser	server-computed only
JSON persistence	file_put_contents(..., LOCK_EX) truncates first	flock around read-modify-write or MySQL
Critical findings	8 open	0
High findings	28 open (5 fixed pending verification)	0 open, or accepted in writing
Route coverage by test	manual	every route render-tested in CI

2. Where the project stands

Measured in the analysis report. The short version:

Strong. 365 server actions across 40 PHP files; 20 MySQL tables / 161 columns; 150 public pages with 142 sitemap URLs; working WhatsApp Cloud API, Telegram bot, Google Analytics 4 + Search Console, Google Sheets sync, MCP endpoint for Claude/Codex/Cursor, and a 25-document audit pack.

Fragile.

Problem	Evidence	Consequence
Four admin generations coexist	admin/ (v1), wf-admin/ (v2 serverless), frontend-v1/admin/ (v2.0), 3 x v2.1	Nobody can say which build is deployed
main does not contain the product	PR #4 -> #5 -> #6 chain never merged	Support answers are wrong by default
JSON side-stores hold secrets and state	28 files under _private/	Non-atomic writes; protection is .htaccess only
Documents are opaque blobs	quotations, invoices, projects, templates all LONGTEXT JSON	Cannot query, index or validate business records
Credentials published	151 files / 36 locations; repository public	8 Critical findings, release recommendation BLOCK
No build step	67 unversioned script tags; load order is the contract	Cannot refactor safely; no cache busting

3. Product definition - what "master dashboard" means

The master dashboard is not a settings page. It is the studio's operating screen and it must answer four questions in under five seconds, for the person who opens it:

1. **What is happening?** - open pipeline value, new leads, won this month, conversion rate.
2. **What needs me now?** - approvals waiting on the master, leads with no reply, failed campaigns.
3. **What is the money doing?** - quoted vs won, average deal size, win rate by service line.
4. **Is the machine healthy?** - site health, backups, integrations, unanswered questions.

Everything else is reachable in one keystroke from the command palette.

3.1 Design principles

#	Principle	Concretely
1	One screen, one job	A screen that cannot state its purpose in one line is two screens

#	Principle	Concretely
2	Density is a setting, not a guess	comfortable and compact change row, control and page height together
3	Never lie about state	No success toast without a request; no total that has not been saved
4	Money is server-side	The browser projects; the server decides
5	Every route is reachable	Registry + aliases + command palette; unknown paths are a real 404
6	Zero literal colour	Components read semantic tokens only
7	A screen is done when it renders for every role, empty and broken	Acceptance gate, enforced in CI

4. Design system - "Woodex Ink" (recommended)

Recommendation: ship both themes with a three-accent switcher, at two densities, driven by a single semantic token contract. Light follows the DESIGN.md direction (white surfaces, blue #2563eb); dark is the obsidian/cyan palette the v2.1 release already shipped (#08090c surfaces, cyan #00d3f2 accent, gold brand mark). Nothing else is per-theme.

4.1 The contract

A component may reference **only** semantic tokens. It may never reference a primitive (ink-800) or a hex value.

```
primitive --ink-900 --cyan-400 --gold-500 <- palette, theme-agnostic
?
semantic --bg --surface --border --text --accent --ok --warn --danger
--viz-1...6 --grid --shadow-* --r-* --s-* --z-*
?
component background: var(--surface); color: var(--text);
```

33 semantic tokens cover the whole product. The prototype implements the full set in admin-v25/src/styles/tokens.css - switch theme, accent and density on the Theme screen and every other screen follows, including the charts.

4.2 Rationale for the choices

Decision	Choice	Reason
Theme	Both, dark default for the studio	Daylight readability for quotes and PDFs; obsidian for long sessions and the projector in a client meeting
Accent	Blue (light) / Cyan (dark), plus Gold and Violet as user options	Blue-gold is the brand; cyan is the product signature the team already recognises
Density	Per user, stored in the account	A quotation desk wants 34 px rows; a designer reviewing images wants 44 px
Type	Inter, 11-28 px, tabular numerals in tables	Already loaded in the current build; numbers must align in columns
Radius	4 / 6 / 8 / 12 / 16	Matches the existing card language so v2.1 screens do not feel foreign
Motion	90 / 150 / 240 ms, honoured prefers-reduced-motion	Nothing decorative; state changes only
Status colour	One statusTone() map for the whole product	Stops "Partial" being green here and amber there

4.3 What this fixes from v2.1

v2.1 defect	Replacement
AD-6 sidebar pinned dark with !important	one --sidebar token, both themes
RC-3 343 literal colours in 21 modules	semantic tokens + a lint rule that fails the build
Charts stayed blue in dark mode	charts read resolved tokens through useTokens()
Token names disagreed (--txt vs --tx)	one published list, checked by the token test

5. Architecture

```
Browser
|
+----------------+-----------------+
| /admin/ React 18 + TS SPA | static, content-hashed, CDN-cacheable
| Vite build - Router - Recharts |
+----------------+-----------------+
| fetch, cookie auth + CSRF header
+----------------+-----------------+
| /api/v2/ PHP 8.2 front ctrl | ONE entry point, JSON envelope
| router -> middleware -> handler |
+---+----------+----------+--------+
| | |
+------+---+ +----+-----+ +--+-------------+
| MySQL | | Storage | | External APIs |
| 20+ tbls | | uploads/ | | WA - TG - GA4 |
| InnoDB | | backups/ | | AI - Sheets |
+----------+ +----------+ +----------------+
^
cron (wa-cron.php) - webhooks (whatsapp, telegram)
```

5.1 Why not other options

Option considered	Rejected because
Keep vanilla JS, add discipline	The failure is structural: mutable global view registry + script-order routing. Discipline does not survive 60 contributors-months
Move to Node/Next in production	Hostinger is shared Apache/PHP. Adds a runtime, a deploy pipeline and a cost, for no user-visible gain
Rewrite the PHP domain too	365 actions encode working business rules. Rewriting them is risk without benefit
Ant Design / MUI	300 kB+ of opinionated components that fight the brand tokens; we need ~25 primitives, which is 600 lines

5.2 Repository layout (proposed, additive)

```
frontend-v1/ <- stays live, untouched during the build
admin-v25/ <- NEW - React + TS source (prototype already here)
src/
lib/ nav.ts - theme.ts - format.ts - api.ts
components/ primitives.tsx - Shell.tsx - CommandPalette.tsx - charts.tsx
screens/ Dashboard.tsx - Leads.tsx - Pipeline.tsx - Quotations.tsx - Theme.tsx ...
styles/ tokens.css - app.css
api/v2/ <- NEW - front controller + handlers
index.php router, envelope, request-id, error boundary
lib/ auth.php db.php store.php caps.php validate.php audit.php http.php
res/ leads.php quotes.php invoices.php clients.php chat.php pages.php ... one per resource
db/migrations/ <- NEW - numbered .sql, forward-only
tests/ <- NEW - route smoke, API contract, migration
```

frontend-v1/admin/ is not deleted until P6 - it is the rollback path.

6. API v2 specification

6.1 Shape

Every response is one envelope. No exceptions, including errors.

```
{ "ok": true,
"data": { },
"meta": { "requestId": "r_8f31c2", "took": 42, "version": "2.5.0" } }
```

```
{ "ok": false,
"error": { "code": "VALIDATION", "message": "Quantity must be greater than zero.",
"field": "lines.3.qty" },
"meta": { "requestId": "r_8f31c2" } }
```

Error codes are a closed set: AUTH, FORBIDDEN, VALIDATION, NOT_FOUND, CONFLICT, RATE_LIMIT, UPSTREAM, INTERNAL. The v2.1 envelope had none of this - admin.php caught only PDOException, so every other throw produced a blank failure with display_errors=0.

6.2 Resource mapping - 365 actions become ~120 endpoints

The rule: **a noun is a resource; a verb becomes a method or a sub-resource.** The v2.1 action names are preserved as an alias table so nothing breaks during migration.

v2.1 actions (examples)	v2 endpoint
leads_list, lead_save, lead_delete, lead_note, lead_convert	GET/POST/PATCH/DELETE /api/v2/leads, /leads/{id}/notes, /leads/{id}/convert
quotes_list, quote_get, quote_save, quote_pdf, quote_view	/api/v2/quotations (+ /pdf, /public/{token})
invs_list, inv_save, inv_new, inv_track, pay_add	/api/v2/invoices, /invoices/{id}/payments
clients_master, client_360, clients_merge	/api/v2/clients, /clients/{id}?include=360, /clients/merge
chat_list, chat_get, chat_reply, chat_typing, chat_file	/api/v2/chats, /chats/{id}/messages, /chats/{id}/typing
pages_list, page_new, page_meta_save, page_delete	/api/v2/pages
cms_list, cms_save, cms_published	/api/v2/content/{collection}
wag_get, wag_camp_save, wag_camp_action	/api/v2/whatsapp/campaigns
seo_list, seo_page, seo_save, seo_scores, seo_robots_save, seo_sitemap, seo_ai	/api/v2/seo/pages, /seo/pages/{rel}, /seo/robots, /seo/sitemap

v2.1 actions (examples)	v2 endpoint
media_list, media_upload, media_delete	/api/v2/media
backup_list, backup_run, backup_restore	/api/v2/system/backups

Full mapping ships as api/v2/aliases.php (action name -> route), generated from the existing dispatcher so it cannot drift.

6.3 Auth - the change that matters most

	v2.1	v2.5
Session	HMAC token in sessionStorage, sent as X-WX-ADM	wx_sid HttpOnly - Secure - SameSite=Lax cookie
CSRF	Builder only	X-WX-CSRF double-submit on every non-GET
XSS impact	token readable -> full account takeover	token unreadable by script
2FA	TOTP, retained	TOTP, retained, plus recovery codes shown once
Roles	owner/admin/editor/sales/support, matrix hardcoded in roles-lib.php + JSON grants	same codes, one seeded capability matrix in db/migrations
Rate limit	wx_throttle by IP	same table, plus per-account lockout and an audit entry

6.4 Persistence rules

1. **JSON writes are atomic.** store.php takes flock(LOCK_EX) on an opened handle, writes a temp file, rename()s it, and reads take a shared lock. jread() returns a typed failure instead of an empty array, so the "Admin is not set up yet" 503 can never be caused by a truncated file again.
2. **Money is server-side.** Totals, tax and rounding are computed in PHP; the browser total is labelled a projection (the prototype shows this on the quotations screen).
3. **Secrets never leave the server.** Connector secrets stay write-only; the browser sees the last four characters, as v2.1 already did correctly in conn-lib.php.
4. **Every write is audited** with actor, action, target, request-id and IP into wx_activity.

7. Module and route plan

All 57 v2.1 routes plus 4 new modules, each with the phase that builds it and the module it replaces. Phase **P0** is the prototype already in this repository.

7.1 Route register (abridged - full table in the prototype registry)

Route	Screen	Phase	Replaces	API
dashboard	Master overview	P0 OK	admin-dash.js + admin-p36.js	GET /dashboard
enquiries / leads	Leads table	P0 OK	admin-crm.js	/leads
pipeline	Stage board	P0 OK	admin-p19f.js	/leads?view=board
quotes	Quotations + BOQ builder	P0 OK	admin-sales.js	/quotations
theme	Design tokens	P0 OK	new	-
approvals	Master approval queue	P2 OK	admin-appr.js	/approvals
clients	Client master + 360	P2	admin-sales17.js	/clients

Route	Screen	Phase	Replaces	API
bookings	Site visits and slots	P2	admin-booking.js	/bookings
invoices	Invoices + tracker	P2	admin-sales.js	/invoices
transactions	Payments	P2	admin-sales.js	/invoices/{id}/payments
chat	Unified inbox	P2	admin-chat.js, admin-chat2.js	/chats
users	Users and roles	P2	admin.js, admin-users.js	/users, /roles
activity	Audit log	P2	admin.js	/activity
profile, security	Account	P2	admin-p18j.js, admin-settings.js	/me, /me/2fa
pages, builder, library, global, heroes, redirects	Website control	P3	admin-pages.js, admin-chrome.js, admin-library.js, admin-hero.js, admin-redirects.js	/pages, /blocks, /chrome, /redirects
blog, portfolio, services, cities, pagetpl, faqs, testimonials, logos, team	Content CMS	P3	admin-content.js, admin-fields.js, admin-templates.js, admin-logos.js	/content/{collection}
media, forms, estimator	Assets and capture	P3	admin-media.js, admin-p18e.js	/media, /forms, /estimator
updates, notify	Client updates	P3	admin-notify.js	/notifications
projects, timeline	Delivery	P3	admin-sales.js, new	/projects
templates	Quote templates	P3 OK	admin-print*.js, admin-sales.js Templates	/quotations/templates (alias of /templates)
business	Company, banks, legal details and the letterhead	P3 OK	admin-fields.js (business), sales-lib.php (company_save)	/business (alias /company) and /business/company
integrations	22 connectors and API keys	P3	admin-settings.js (rewritten from hard-coded cards)	/connectors
seo OK (brought forward into P3), speed, health, seoagent	Findability and quality	P4	admin-seo.js, admin-seo38.js, admin-speed.js, admin-media.js, admin-seoagent.js	/seo, /speed, /health, /seo/agent
aicenter, train, aireport	AI control	P4	admin-aicenter.js, admin-train.js, admin-aia.js	/ai, /ai/knowledge, /ai/report
whahub, wainsights, wauto, offers, connections	WhatsApp suite	P4	admin-wahub.js, admin-wains.js, admin-p18g.js, admin-offers.js, admin-p38wa.js	/whatsapp/*
telegram, social, gmail	Other channels	P4	admin-tg.js, admin-social.js, new	/telegram, /social, /mail
backups, database, files, maintenance, system	System	P5	admin-media.js, admin-p18h.js, admin-system.js	/system/*
inventory, procurement, portal	v2.5 additions	P6	new	/inventory, /purchase-orders, /portal

7.2 Navigation target - 6 groups -> 8 groups

Group	Routes	Change from v2.1
Home	3	+ Timeline (Gantt)
Sales	11	+ Inventory, Procurement, Client portal
Conversations	10	+ Mail
Website	18	unchanged, regrouped by intent
Automation	4	new group - AI, knowledge, report, SEO agent moved out of Website
System	12	merged Business + Integrations + maintenance
Me	2	unchanged

Group	Routes	Change from v2.1
Total	60	57 -> 60

8. Dashboard content plan

8.1 Widgets and their data source

#	Widget	Source	Refresh	Notes
1	KPI row (4 cards)	GET /dashboard?range=	route load + 60 s poll	Role-aware: money hidden from support
2	Quoted vs won area chart	dashboard.series	route load	12 months, server-aggregated
3	Needs attention feed	computed server-side	60 s	approvals, silent leads, failed campaigns, health regressions
4	Channel mix bars	dashboard.channe	route load	website, WhatsApp, chat, Telegram, phone
5	Service mix table	dashboard.services	route load	count, value, share
6	Latest quotations	/quotations?limit=6	route load	status pills via statusTone()
7	Studio activity	/activity?limit=6	60 s	every write is already audited
8	Quick actions	static	-	six most common studio tasks
9	Approvals banner	/approvals?count	60 s	only for owner/admin/editor
10	Health strip (P4)	/health	daily	score, findings, last scan

8.2 The "needs attention" rule engine

This is the widget that makes the dashboard a tool rather than a report. Each rule is a SQL query with a threshold in wx_settings, so the owner can tune it without a deploy:

Rule	Condition	Severity
Silent lead	stage = new and age > 24 h	amber
Overdue follow-up	next_at < now and stage not won/lost	red
Quotation viewed, not answered	status viewed and 3 days old	amber
Approval waiting	pending > 8 h	amber
Campaign rejected	Meta template status ? approved	red
Backup stale	last backup > 48 h	red
Site health drop	score < previous ? 5	amber
Unanswered AI question	same question x3 and status open	amber

9. Database plan

9.1 What exists (20 tables, 161 columns)

Full schema in the analysis report Section 7. The three structural problems to fix:

1. **Financial documents are opaque.** wx_quotes, wx_invoices, wx_projects, wx_templates store payloads as LONGTEXT JSON, so the database cannot index, query or validate the substance of the business's money.
2. **Authorisation is code, not data.** roles-lib.php hardcodes the entire capability matrix (wx_role_groups(),

wx_action_group(), SHARED, EXPAND) and per-user grants live in the JSON side-store _private/security.json, so permissions cannot be inspected, audited or changed without a deploy. *Correction:* an earlier revision of the analysis report called these five *tables* created at runtime and missing from the dump. They are PHP **functions**; the dump's 20 tables are complete. The matrix is still the wrong shape - 001_roles.sql moves it into seeded tables.

3. **wx_leads has 32 columns** and no foreign keys anywhere - integrity is application-only.

9.2 Migration set (forward-only, numbered, one transaction each)

#	Migration	Purpose
001_roles.sql	Move the capability matrix out of roles-lib.php into seeded tables (wx_roles, wx_role_groups, wx_action_groups, wx_user_perms); seed the 5 roles x 9 groups matrix	Closes DB-1
002_indexes.sql	Composite indexes: leads(stage, created_at), leads(next_at, stage), chats(status, updated_at), invoices(no), quotes(no, status), bookings(d, status), activity(created_at, user_id)	Fixes the N+1 and full-scan behaviour reported in the audit
003_document_columns.sql	Add real columns beside the JSON blobs: wx_quotes(client_id, project, subtotal, tax, total, currency, status, issued_at, valid_until, pdf_sha256), same for invoices; backfill from JSON with a one-way script; keep the JSON as payload for compatibility	Makes money queryable
004_fk.sql	Add FKs after cleaning orphans: lead_notes.lead_id -> leads.id, chat_msgs.chat_id -> chats.id, invoices.quote_id -> quotes.id, bookings.lead_id -> leads.id (all ON DELETE CASCADE or SET NULL as appropriate)	Integrity at the database, not in code
005_settings_seed.sql	Seed wx_settings with the attention thresholds and UI defaults	Config, not code
006_inventory.sql	wx_items, wx_stock, wx_stock_moves, wx_suppliers, wx_purchase_orders, wx_po_lines	P6 inventory + procurement
007_portal.sql	wx_portal_tokens, wx_client_approvals, wx_client_uploads	P6 client portal
008_mail.sql	wx_mail_threads, wx_mail_messages	P6 shared mailbox
009_versions.sql	wx_page_versions (page, html, note, created_by, created_at)	P3 page history - the plan already references versioning
010_json_retire.sql	Move the remaining 28 _private/*.json stores into wx_kv for the small ones and real tables for the important ones (approvals, whatsapp opt-outs, notifications, redirects, seo)	Ends the non-atomic-store class of bug

9.3 Target size

	Today	After P6
Tables	20	38
Columns	161	~330
Foreign keys	0	14
JSON side-stores	28	0 for business state (5 remain for caches only)

Data safety. Every migration is applied to a restored copy first, with a row-count and checksum comparison before and after, and a dry-run mode. Nothing drops a column in the same release that stops writing to it.

10. Security remediation

The 8 Critical findings must be closed **before** any v2.5 code is deployed. They are listed in order of consequence, not effort.

#	Action	Owner	Verification
1	Rotate the shared demo password (admin account, DB user, any derived key)	Owner	Old value fails on the live host
2	Rotate the Telegram bot token in BotFather	Owner	Old token returns 401 from the Bot API
3	Rotate the application key and maintenance token	Owner	/coming-soon no longer accepts the old token
4	Make the repository private	Owner	gh repo view -> isPrivate: true
5	Remove the login pre-fill and the empty-password fallback (admin/index.html:43, admin.js:235,256)	Dev	Search returns zero hits
6	Remove the admin sign-in hint from downloads.html:70	Dev	Page reviewed
7	Untrack .env, config.php, _private/*; add .gitignore to the pro tree	Dev	git check-ignore lists them
8	Verify private-path 403/404 on the live host, then move state out of JSON (migration 010)	Dev	curl returns 403 for each path

Then the High findings, in P1 and P2:

Finding	Fix	Phase
Non-atomic JSON writes (jwrite truncates before locking)	store.php with flock + temp file + rename	P1
HMAC secret regenerates on empty read, invalidating all sessions	secret stored in MySQL with a checksum; regeneration logged as a Critical event	P1
WhatsApp cron takes a LOCK_NB and admin writers take none	one lock helper used by both; busy becomes a visible 409	P4
Only PDOException caught	an error boundary in the front controller: every throw returns a coded envelope and a request-id	P1
wx-check.php / wx-install.php shipped in production	installer removed from the release artifact; check restricted to owner	P5
143 images without alt text	media library shows a missing-alt queue; health scan links to it	P3

11. Delivery phases

Each phase ends with a **gate**: a written, testable condition. A phase that fails its gate does not start the next one.

Phase 0 - Foundation and prototype - **complete** - 1 week

Deliverable	Status
Token contract + two themes + accents + densities	OK src/styles/tokens.css
Typed route registry (65 routes, aliases, roles, phases)	OK src/lib/nav.ts
App shell: sidebar, groups, topbar, breadcrumbs, CmdK palette	OK
Five screens: dashboard, leads, pipeline, quotations, theme	OK
Render smoke harness over every route	OK 70 checks - 65 routes - 0 failures

Deliverable	Status
Production build with vendor chunking	OK 80 kB app / 160 kB vendor / 388 kB charts

Gate: npm run verify passes - typecheck, 65-route render test, production build.

Phase 1 - Platform - **built** - 2 weeks

Work	Deliverable	Status
api/v2 front controller	index.php + lib/{bootstrap,errors,http,store,db,validate,audit,auth,caps,router}.php - router, one envelope, closed error set, error boundary, request-id, CORS-free same-origin	OK 19 files, PHP lint clean
Auth v2	wx_sid HttpOnly - Secure - SameSite=Lax cookie, X-WX-CSRF bound to the session row, 2FA + recovery codes preserved, capability matrix in tables	OK
store.php	JsonStore - sidecar lock, temp+fsync+rename, a corrupt store raises instead of reading as empty	OK
Client api.ts	typed fetch, closed error union, request-id on every failure, timeout, retries on GET only, token in memory	OK
db/migrations/000-005	platform tables, generated role matrix, 12 indexes, document columns, 10 FKs, settings seed - with db/migrate.php --dry-run/--status/--verify, and db/owner.php for the account that can sign in	OK
CI	npm run verify + PHP lint + conformance + behavioural tests + migration dry-run	OK .github/workflows/verify.yml

Gate: a signed-in request round-trips cookie -> CSRF -> controller -> MySQL -> envelope; the eight Critical findings are closed and verified.

Verification	Command	Result
PHP parses (no runtime in the build sandbox)	node tools/phplint/php-lint.mjs	103 files clean
Structural invariants (47 checks: auth on every route, envelope discipline, lock discipline, cookie flags, SQL safety, migration guards, the 8 Criticals)	node tests/conformance.mjs	47 passed - 0 failed
Behaviour (84 assertions: paisa money, validation, RFC 6238 TOTP vectors, envelope, aliases, routing, atomic store)	node tools/phpwasm/run-php.cjs tests/run.php	84 passed - 0 failed - 1 skipped
The gate itself, against MySQL	php tests/roundtrip.php	written; runs in CI, needs a database
Prototype	npm run verify	70 render checks - 65 routes - build clean

The two-sided verification is deliberate: tools/phpwasm/run-php.cjs executes the real PHP in the build sandbox (a WebAssembly 8.2 build - the ESM/CJS trap that makes it fail has a note in tools/phpwasm/README.md), and the MySQL leg runs in CI where a real server exists.

Also closed in this phase: the capability matrix moved out of roles-lib.php into seeded tables (tools/gen-caps.mjs regenerates them from the file v2.1 enforced, so the answers cannot differ); the public demo-credential page and the admin/admin installer default are gone; findings 5-8 have executable checks rather than a claim.

Phase 2 - Core operations - **built** - 3 weeks

Approvals - clients - bookings - invoices - payments - unified inbox - users and roles - activity - profile - security - leads and pipeline and quotations **port from the prototype to real data**.

Gate: a lead can be created on the website, appear in the inbox, be quoted, approved, invoiced and paid without touching the v2.1 admin. Money totals match the server.

Built	Where
Money, documents, totals, quote-view tokens, receipts	api/v2/lib/{money,docs}.php - the invoice's displayed total is the stored total
Approvals (the policy copied from v2.1 and verified as a copy)	lib/approvals.php + routes/approvals.php
Bookings, clients, projects, quotations, invoices, payments	routes/{bookings,clients,projects,quotations,invoices}.php
Unified inbox (widget, WhatsApp, Telegram shapes), the bell	routes/chats.php, routes/chat_public.php, lib/notify.php, routes/notifications.php
Users, roles, 2FA, recovery codes, sessions	routes/{users,security}.php + lib/auth.php
Leads, pipeline, dashboard, me, activity, system, health	routes/{leads,dashboard,me,system}.php
The dashboard that talks to it	admin-v25/src/lib/live.ts, screens/{SignIn,LiveDashboard,LiveLeads}.tsx

Verification	Command	Result
PHP parses	node tools/phplint/php-lint.mjs	127 files clean
Structural invariants	node tests/conformance.mjs	69 passed - 0 failed
Behaviour	node tools/phpwasm/run-php.cjs tests/run.php	254 passed - 0 failed - 4 skipped
The gate itself, against MySQL	php tests/roundtrip.php	written; runs in CI, needs a database
The admin	npm run verify	78 render checks - 65 routes - build clean

Coverage: 150 of the 365 v2.1 actions are served by api/v2 today and the rest answer NOT_FOUND naming the action and the v2.1 file that still serves it - the strangler pattern working as intended, not a gap hidden by a fallback.

Phase 3 - Content, assets and delivery - **in progress** - 4 weeks

Measured at this commit: tests/run.php **1222 passed - 0 failed - 14 skipped** - tests/conformance.mjs **86 - 0** - php-lint **161 files** - admin-v25 smoke **155 checks - 65 routes** - legacy bridge **158 of 365 actions** (43%). Migrations **000-014** are written and none has run on a database yet.

Three generators are part of this phase's gate rather than a convention, because each one replaces a second hand-typed copy of something the site already owns: tools/gen-aliases.mjs check (365 actions), tools/gen-caps.mjs check (the permission matrix) and tools/gen-quote-starters.mjs check (the nine starter BOQ templates, extracted from assets/js/quotation-templates.js - the file the website loads).

Pages - builder bridge - section library - header/footer - heroes - redirects - the full content CMS - media with alt-text queue - forms - estimator - projects and the Gantt timeline - client updates - quote templates - business info - integrations rebuilt on real connector state - page versioning.

Gate: an editor publishes an article, changes the header across 150 pages, and rolls the change back from page history - all through v2.5.

Delivered	Where	Note
Redirect rules as rows, the 404 log, a safe .htaccess publish	db/migrations/009_redirects.sql, lib/redirects.php, routes/redirects.php	the rules are validated as a whole and published from that same list; a copy of the file precedes every write

Delivered	Where	Note
Pages, versioning and rollback	db/migrations/010_media_pages.sql, lib/pages.php, routes/pages.php	one batch per change, so the header across 150 pages is one undo
Media library: usage, derivatives, alt-text queue, trash	lib/media.php, routes/media.php	alt text and the check that it was looked at are recorded separately
File manager over assets/	lib/files.php, routes/files.php	four shared folders read-only; a file a page uses is refused before it is renamed or deleted
The header, the menu and the footer across every page	lib/chrome.php, routes/chrome.php, db/migrations/011_chrome.sql	one edit for the whole site, keeping what is per-page: the current page's own link, the home page's brand, the mobile button's target, each page's footer anchors and its own intro sentence. A check that writes nothing and an apply that returns the batch id, so the header across 150 pages is one undo
The content CMS: an editor writes an article, the server composes the page	lib/content.php, routes/content.php, db/migrations/012_content.sql	v2.1's builder ran in the browser and posted finished HTML; here an item is content and the page is composed at publish time from the site's own page furniture - so a new article inherits the current header, scheduling is a row with a time rather than 3 MB of prepared HTML, and drafts are a query rather than a loop
Eight live screens, ten routes	admin-v25/src/screens/Live*.tsx on /redirects, /media, /blog, /portfolio, /global, /pages, /library, /site-theme, /testimonials, /team	live-only by design; with no API they say so in a sentence and offer no buttons that would lie. /blog and /portfolio are one screen (the CMS, opened on a kind), and so are /testimonials and /team

What a screen may claim when the server is not there. Four states are visible rather than hidden behind a spinner: the API is being looked for, it is not answering, it is answering and you are not signed in, or you are signed in. The first one is the state nobody could see - the door used to render a live Sign in button against a server that had not answered yet - so it now says it is still looking and stays closed. The third one is where a prototype lies by habit - it falls back to mock data under a green label. Here the shell names the state - its pill reads live, checking, signed out, not answering or demo data, and the pill and the sentence beside it are derived from the same two facts so they cannot disagree - and each screen then says only what is true about *itself*: the sign-in screen shows no site data whatever, so it says there is nothing behind the form to open a session rather than telling the person they are "looking at the UI with demo data" - a sentence that was true of the prototype with a mock behind it and false of a real door with a disabled button. The words "demo data", and the label naming the v2.5 Pro plan, appear in **demo mode only**: an admin that cannot reach its own host is reported as an unreachable host, because calling that a demo tells the person their console is a mock-up when it is really a door that cannot open yet. A screen that cannot load its data offers no button that would pretend to work, and the smoke test asserts both halves of that.

All three sentences of the gate are now wired, and the wiring is asserted (tests/run.php, "the gate, as wiring"): POST /content/{id}/publish composes and writes the article and returns the batch that undoes it; POST /chrome changes the header across the site through the same versioned writer; GET /pages/history groups the result as one change and POST /pages/rollback puts every page in it back - the undo is itself a batch, so it can be undone. Running the gate itself needs PHP and MySQL: **migrations 000-012 have still never run**, and there is no PHP in the build sandbox, so the gate is a host run on the checklist above.

Delivered - the history screen	Where	Note

Delivered - the history screen	Where	Note
Every page, every change, one undo	admin-v25/src/screens/LivePages.tsx, lib/pages.php#wx_page_changes_grouped()	the change that rewrote 148 pages is one entry with one button; per-page versions sit beside it
The section library, and global sections pushed from one edit	lib/blocks.php, routes/blocks.php, db/migrations/013_blocks.sql, admin-v25/src/screens/LiveLibrary.tsx	a block is a piece of a page; a global block is inserted by reference - the page carries its token - so editing it and pushing rewrites every page that uses it as one batch with one undo. The token is 12 hex and lives in the HTML files, not the row id, which is why a database restore cannot orphan a section
The page editor, and the script rule that guards it	lib/pageedit.php, routes/pages.php (POST /pages, POST /pages/new), the editor panel in LivePages.tsx	v2.1's audit finding F-05 kept as a rule about additions: only the owner may add scripts, embeds or on...= handlers to a page, anybody may keep what is there, and a tag the site already uses elsewhere is the site's. Tokens are compared after entity-decoding and whitespace-stripping, so javascript: is the same token as javascript:. Plus the optimistic-concurrency rule: a page that changed on the server since it was opened is refused with a 409 rather than overwritten

Delivered - the site theme	Where	Note
v2.1's Theme settings, written into the stylesheet the site actually loads	lib/theme.php, routes/theme.php, admin-v25/src/screens/LiveSiteTheme.tsx on /site-theme	v2.1's builder wrote assets/theme.css and every page linked it. This site has no such file - the live pages link /assets/site.css and nothing else - so the port keeps the design surface (the same list of variables, the same validation, value for value) and changes only where the values land: two marked blocks in that stylesheet, the Google-font @import at the top where the spec allows it, the values and their aliases at the end where they win by order. Every version is a normal entry in wx_page_versions keyed by assets/site.css, so the theme is covered by the same history and the same undo as the pages, and clearing every setting removes the blocks and gives the file back byte for byte

Delivered - where the sections appear	Where	Note

Delivered - where the sections appear	Where	Note
Testimonials and the team, placed inside pages	db/migrations/014_placements.sql, lib/placements.php, routes/placements.php, admin-v25/src/screens/LivePlacements.tsx on /testimonials and /team	These two kinds never become pages: they are written inside them, between v2.1's own markers (<!--wx:testimonials--> ... <!--/wx:testimonials-->), which is what lets a push replace yesterday's section instead of adding a second one - and what removes it when a page comes off the list. The studio's choices travel value for value from v2.1 (cms_placements: the pages per kind, the small label, the heading, a maximum, and the five testimonial designs) and the section is composed on the server by the same function that writes it, so the preview is the push rather than a mock-up beside it. Adopting this landed two things v2.1 could not do: the push writes every changed page in one batch (v2.1 posted one page per request from the browser, so a failure halfway left the site in two states; here the whole push is one entry in the page history and one undo), and a page is refused rather than guessed at when it has no anchor to put the section in
The section designs, in the stylesheet the site loads	assets/site.css	The wx-tst-, wx-team-, wx-stars and wx-kicker rules existed only in a second stylesheet the live pages do not link, so a section written without them would have landed unstyled - the same failure shape as the removed theme.css, in the other direction. They now sit in assets/site.css, reading the theme's --wx-* values where a theme has been saved and this stylesheet's own tokens where it has not

Delivered - the builder bridge, complete	Where	Note
The last of v2.1's 19 builder actions	api/v2/routes/legacy.php (wx_legacy_params()), lib/pages.php (wx_page_backup_*), routes/pages.php (GET /pages/backups)	v2.1 names a page path and the route that reads it says rel, so the bridge now translates parameter names instead of hoping they match - the old page screen loads and saves through /api/v2 again, and a missing parameter names the key the caller used. Two of the three left were bugs of exactly that shape. backups answers with both stores, each labelled: the copies v2.1 saved before every change (validated as names, readable and restorable by restore {path, file}, going back through the page writer so a restore is a versioned change with an author and an undo) and the versions this API has written since. Two new conformance checks keep it honest: every bridge entry's handler exists in the file it names and in the route table - a renamed handler is a 500 for whoever called the old name, and nothing else catches it - and the parameter aliases are asserted to be present rather than incidental

Delivered - the builder bridge, complete	Where	Note
The two that are decisions, not gaps	media_delete, media_usage	Neither is mapped, and the reason is in the code next to them. Deleting an image in v2.5 is a trash with a 30-day window that refuses to touch a file a page uses, so answering v2.1's name with a trash call would teach the caller something untrue about what just happened to their file; and usage is not an action in v2.5, it is a field of the library read. Both keep working through /api/builder.php until the screens that use them are gone

An audit of the approval policy, and what it found. Checking that every write route is gated is not enough - a gate whose action the policy would never queue is decoration. Two directions are now asserted in tests/conformance.mjs: every non-GET route either calls the gate or matches WX_APPR_SKIP, *and* every gated route's action either queues or appears in a new WX_APPR_NEVER map with the reason written next to it. That found one real hole: POST /pages carried the invented action page_write, so a Manager could edit any of the 150 pages with no approval while needing one to change a section block. The routes that came out of the builder now carry the builder's own action names (save, restore, page_new, theme) and the gate is told the *source* is builder, which is the pair v2.1's policy is keyed on. Four actions are gated on purpose and never queue, each with its reason in the policy: a rollback is an undo of a change that already passed, publishing an article never queued in v2.1, regenerating image derivatives changes nothing anybody reads, and a draft revision of a quotation is a copy. One deliberate tightening is recorded as such: merging the 36-address redirect plan now queues under redirects_save, because typing the same rules by hand already did.

Delivered - the two fetchers	Where	Note
v2.1's fetch_page and import_url, with the SSRF rules that came with them	lib/fetch.php, routes/pages.php (POST /pages/fetch), routes/media.php (POST /media/import)	A server-side request to an address the user pasted is the definition of SSRF, and v2.1's audit F-06 grew a set of rules for it - https only, no credentials or ports in the URL, a name rather than a numeric address, every resolved address public, every redirect hop re-checked, 3 MB of HTML and 8 MB of image, the certificate and the name verified. All of them are ported, one hardening is added on top (::ffff:127.0.0.1 is reserved rather than private, so PHP's own flags pass it), and the transport is injectable - so the refusal that matters, a public link redirecting to 169.254.169.254, is asserted against a fake transport rather than hoped for. The image lands through the same wx_media_upload() the upload button uses, so what may be stored is decided once

Delivered - city pages	Where	Note

Delivered - city pages	Where	Note
A city page is root-level (/<address>/), and its furniture comes from a template rather than from a copy someone edited	wx_content_rel(), wx_content_shell_rel() in lib/content.php, the /cities route in App.tsx	v2.1's model was copy an existing city page and rewrite each string - 150 pages of drift with a person in the middle. Here the address and the furniture are two rules: one function returns /<slug>/index.html for a city and /<section>/<slug>/index.html for a post or a case study (so the address is answered in one place for the item, the publish and the tick), and one function decides which page the furniture is read from - the template's source city if it names one, otherwise the front page, with content.shell.city to override both. The guard list is v2.1's (insights, projects, the machinery folders), and it is applied twice: when the address is typed and again at publish, because an item imported from v2.1 never passed through the form. Eleven assertions cover the rule itself; the publish path needs a database

Delivered - page heroes	Where	Note
The hero at the top of every page, and the home page's slider: read as geometry, edited in place, or switched to one of six designs	lib/heroes.php, routes/heroes.php, LiveHeroes.tsx on /heroes, assets/site.css, assets/site.js	v2.1's last big screen that lived only in a browser. It read the page into the DOM, edited nodes and posted the whole document back - and a field with no node in that page's hero (a kicker a design does not have) was silently dropped. Here the geometry is computed first: every field's text or attribute is a byte range, and the new values are spliced in, so a field the request wants to set that has no home is refused by name, and nothing else in the page is touched because nothing else is written. The reader and the writer share one function for the same reason - the bytes a field is read from are the bytes it is written at. Six templates (data-wxh) for a page whose hero should be the studio's own design; the gallery draws each one with sample words through the same renderer that writes a page. Both writes go through the page writer, so a hero change is one version and one undo

Delivered - the website's forms	Where	Note

Delivered - the website's forms	Where	Note
Six forms: which pages carry one, where each page's enquiry actually goes, and the alert settings	lib/forms.php, routes/forms.php, LiveForms.tsx on /forms	v2.1's screen looked for its own markup (WXForms.send('contact'), data-form="...") to say which pages carried a form. On this site that answer is worse than useless: the pages post their enquiries to Netlify functions, so the screen would have shown six forms, six zeros and no reason. So the read names the address each page posts to and whether that address is one whose leads land here - a form going elsewhere is labelled as such, by address. A page's form is found by the id it sends or by the id its own markup carries (read off the live pages: est-form, brief-form, fitout-hub-form, fitout-quote-form), and the WhatsApp widget is described as what it is: loaded by assets/site.js on every page, posting nothing

Delivered - the rate book	Where	Note
The figures /estimator/ and the quotation builder price from, edited in one place	lib/forms.php, routes/estimator.php, LiveEstimator.tsx on /estimator	Kept v2.1's shape on purpose: the book is written out as JavaScript (assets/js/estimator-rates.js, window.WX_RATES) because the public page reads it with no request - which is why the calculator works on a phone with one bar of signal. The rules a quote can be wrong by are checked and named (unique keys, rates above zero and rising Essential <= Standard <= Premium, a catalog line without a description dropped), and the reload of every reading page is one wx_page_write_many() batch - where v2.1 rewrote each page with a file write of its own, outside the page history. Saved sets (v2.1's P19 templates) keep a whole book under a name

Delivered - projects	Where	Note

Delivered - projects	Where	Note
Every project, its stage and its money, with a drawer that moves the stage, edits the milestone plan and bills one	routes/projects.php (wx_proj_row(), wx_proj_filter(), wx_proj_kpi()), admin-v25/src/screens/LiveProjects.tsx on /projects	Two numbers a project has two answers to - where it stands among the stages, and how much of the contract has been billed - were one field in v2.1 and are two named ones here (stageProgress, billedPct), because a single progress printed as both is how a screen ends up half right. The money fix: a project can be billed two ways (one invoice for the contract, or one per milestone), and the v2.5 list route originally read only the first - a project billed in three milestones reported invoiced nothing. Both link shapes are read now, and the assertion that covers it is the one that would have caught it. The six figures at the top come from SQL aggregates over every project, not from the page on screen: a contract value totalled from the fifty rows a table happens to hold is a lie the moment there are fifty-one, and v2.1 read all of them into PHP to avoid it (the plan's 1.4-second table). Milestone statuses keep three facts apart - billed, partly paid, and invoice deleted - because a milestone whose invoice vanished must not look unbilled

Delivered - the timeline	Where	Note
Every project on a date axis, from its start to its handover, filled by what has been invoiced	admin-v25/src/screens/LiveTimeline.tsx on /timeline, reading GET /projects?view=all	The plan asks for a Gantt; the data holds two dates per project and a milestone plan that is percentages of the contract, not dated tasks. So the chart draws what exists - a bar per project, a today line, late in red - and says so, rather than inventing a task schedule nobody agreed to and somebody promising a handover from. A project with no dates cannot be a bar, so it is listed underneath with its stage and money instead of being dropped: a chart that silently omits work looks complete and is not. The axis spans the work's own range (extended to today, so "everything is behind us" is visible), the label column stays fixed while the track scrolls, and each zoom level names its unit

Delivered - client updates	Where	Note

Delivered - client updates	Where	Note
The seven events and their bilingual wording, the channels, the log - and the page the client opens	lib/notify.php, lib/clientview.php, routes/notifications.php, routes/client_view.php, LiveUpdates.tsx on /notifications, the drawer on /projects	Two halves, and the honest one is the smaller. The page: /api/v2/client/project/{id}?t=... - an HMAC of the project's id and number, the construction v2.1's quotation link has always used, compared with hash_equals, no session, no script tag, and the first view recorded once every twelve hours so the studio can see the link was opened. It renders updates[] entries marked toClient and nothing else: one record, so there is no second copy of a sentence to drift and no "what did we actually tell them?". The sending: v2.5 has no WhatsApp or SMTP transport, and the plan puts it in Phase 4 with the WhatsApp suite - so the send path reports no transport yet rather than v2.1's queued, the once-only stage marks are set only when a message left, and the screen says who sends today instead of implying it is us. That last part is not decoration: marking a stage "told" on the strength of a log row would silence that news for good the day the channels are connected. A stage message has two shapes - the photo links, or, when a stage has no photos yet, the same words pointing at the client's page - and the update a person types in the drawer is the studio's own until they tick Show the client

Delivered - quotation templates and the document	Where	Note

Delivered - quotation templates and the document	Where	Note
The nine BOQ templates the studio has quoted from for years, the editor for the section tree, and the quotation a template produces - drawn, not described	lib/templates.php (wx_tpl_clean(), wx_tpl_row(), wx_tpl_starters()), routes/quotations.php (wx_route_tpl_*), api/v2/data/quote-starters.json, tools/gen-quote-starters.mjs, LiveTemplates.tsx on /templates (alias /quotations/templates), components/DocPreview.tsx	v2.1 kept templates in wx_templates as JSON blobs and printed from a browser script (admin-print17.js) that built the page with document.write. Four things were wrong with that, and each one is a decision here. The content existed twice - nine templates, eighty line items - as the site's assets/js/quotation-templates.js and as whatever somebody had typed into the admin; the shipped set is now generated from the site's own file (node tools/gen-quote-starters.mjs), and a conformance check re-runs the extraction, so a rate edited in one copy and not the other fails the gate instead of becoming a quotation with the old price in it. A template was only as valid as the browser that wrote it: sections now go through wx_clean_sections(), the same cleaner a quotation's own sections go through (a line with no description is refused by field, a unit outside v2.1's list becomes job, a heading carries no numbers), and the total is derived by wx_doc_totals() - the one implementation of document arithmetic. Imports overwrote by name: existing names are skipped, never replaced, and the reply names what was added, skipped and refused, because "9 imported" when six were skipped is how a list looks wrong on the next load. Nothing but a browser could produce a quotation: the document is now a component (DocPreview) that computes nothing - every figure arrives from the server - keeps v2.1's numbering exactly (headings 5.1, their lines 5.1.1, specification lines unnumbered, a hand-numbered line keeps its number), and carries the three designs (Classic, Minimal, Premium) as one set of variables rather than three copies of a CSS string. Saving a template runs through the same approval gate as a quotation, with labels the Master reads in words ("Save a quotation template: Renovation")

Delivered - demo mode, so a preview shows the console	Where	Note

Delivered - demo mode, so a preview shows the console	Where	Note
The screens built in this phase, with sample data behind a label that cannot be missed	admin-v25/src/lib/demo.ts (fixtures keyed by API path), useLive(), DemoDoor/DemoBanner in App.tsx	A console that reads a real API shows nothing at all where there is no API - a laptop, a sandbox, a hosted preview - and the one thing a preview must never do is let somebody think they are looking at their own figures. So demo mode is asked for (?demo=1, or the door on any screen that is saying the host is not answering) and never inferred; the shell says demo data; a banner above every screen says no part of it came from the server and offers the way out; and a screen whose data nobody has invented yet keeps its honest "not answering" notice, which is how the next slice announces itself. The fixtures are typed against the same interfaces the server's answers are, so when a shape changes the sample stops compiling rather than quietly becoming a nicer product than the product. Also fixed here: a failed GET /me used to turn a settled "host not answering" into a sign-in form - a door that cannot open, inviting a password into nothing

Delivered - business info, read off the site instead of out of a copy of the defaults	Where	Note	
The phone number, the address, the hours and the socials on every page, and the letterhead every document prints	api/v2/lib/business.php (wx_biz_, wx_company_), api/v2/routes/business.php (/business, /business/company), admin-v25/src/screens/LiveBusiness.tsx on /business (alias /company)	v2.1 answered "what does the website say?" out of its own stored copy of the defaults - on this site, whose footer opens at 9:30 while that record said 10:00, every replacement matched nothing and the screen still reported success. The baseline is the site now: what a real page says, or what was last applied, and the answer carries source so the screen can name which one it is showing. The edit goes through the pair engine as one batch that returns its own undo (POST /pages/history/{batch}/rollback), with the WhatsApp number swapped through a sentinel so the digits-only phone pairs can never paste the office number into a wa.me link, and pairs deduped by from\0to. The second half is the letterhead documents print from: banks, wallets, signers and the next quote/invoice number, refused when it is at or below the highest number already used	
Delivered - the announcement bar, and both halves of the site that draw it	Where	Note	

Delivered - business info, read off the site instead of out of a copy of the defaults	Where	Note	
The thin bar above the header - on or off, the words, an optional link, three colours - the same file v2.1's site read	api/v2/lib/announce.php, api/v2/routes/chrome.php (`GET\	POST /chrome/announce), admin-v25/src/screens/LiveGlobals.jsx (Header & footer -> Announcement bar), assets/site.js + assets/site.css (the two halves), assets/announce.json` (runtime state, not in git)	The feature was working in the older copy of this site and absent from the files index.html actually links, so the console could save an announcement no page could draw - the exact shape of "the screen says saved and nothing appeared". Both halves ship here, re-aimed at this design: the header is position:fixed, so the bar offsets it by its own 36px (and the mobile menu with it) rather than pushing a flowing header down with a margin. The screen reports which half is missing, and how many pages load the script, instead of promising a bar. The write is the atomic JSON store - v2.1's file_put_contents(..., LOCK_EX) truncated before it locked, so a page fetched during a save could get half a file and show nothing - the version moves only when the announcement changed, because it is the key a visitor's dismissal is remembered under, and the record is v2.1's six keys so the older site still renders the same file. A conformance check runs the shipped block against a stub browser and asserts the bar it puts on the page

Delivered - the AI settings, and the first call this API makes outward	Where	Note	

Delivered - the AI settings, and the first call this API makes outward	Where	Note	
The four providers, their keys and models, the writing voice, the model list, and one small test call	api/v2/lib/ai.php (wx_ai_*), api/v2/routes/ai.php (`GET\	POST /ai/settings, POST /ai/models, POST /ai/test), admin-v25/src/components/AiSettings.jsx at the foot of the Content screen	The record lives in v2.1's own file (_private/content.json), so things still admin and the new one read the same settings while the strangler runs - and a save carries the rest of that file through untouched. Four things v2.1 did are each a decision here: a read could leak a key, so wx_ai_pub() is now the only serialiser and the tests assert the key appears nowhere in the answer; an empty key field meant erase, so absent and empty both mean keep and only <k>Clear deletes - with a reason, because a form that wipes keys when somebody fixes a typo in the voice is a form that loses the studio's keys; clip($v, 300) shortened a mistyped key into a shorter, wrong one that failed later with a provider error, so this refuses by field; and the request, the response and the model filter sat inside one function that only ran with cURL and a key, so they are three pure functions now - which is why they are tested at all in a sandbox with no cURL, no key and no route to a provider. What that leaves untested is one curl_exec() and the honest report of it
Delivered - publishing a page, and the service and city views	Where	Note	
Whether a page is public, and everything that follows from it: the head's tags, one word in _private/pages.json (v2.1's file), the sitemap, and a screen that shows all three	api/v2/lib/pages.php (wx_page_head_plan() pure - wx_page_publish() - wx_page_state() - wx_page_sitemap_set()), POST /pages/{rel}/publish (action page_meta_save), the bridge row in routes/legacy.php, the card on admin-v25/src/screens/LivePages.jsx	v2.1's page_meta_save, kept under its own name because the Master's queue reads it, with one repair: v2.1 had sitemap_add() and nothing that took an address out, so "unpublish" went on telling Google the page was current - the sitemap now follows the status both ways. The rules are one pure function, which is the only reason they are tested at all in a sandbox with no database, and a new page is still saved as a draft: a page that is public the moment it is created is a page in the sitemap before anybody has looked at it	

Delivered - the AI settings, and the first call this API makes outward	Where	Note	
The service and city views, built from the pages the site actually has	the same screen, over /content/pages - every page's own <body data-page>	v2.1's services screen was never a collection: it was a view of the pages (kind ? city, minus articles, studies and the machinery), and cities the same list the other way round. The kind is read from the page rather than decided by the screen, and with no answer the screen shows every page and says so. The city content model - the address rule and the furniture rule - is the plan's own model, already delivered	

Delivered - findability, and the working out moved to the server	Where	Note		
Every page audited against the rules, the phrase each page is aiming at, its structured data, robots.txt and sitemap.xml	api/v2/lib/seo.php, api/v2/routes/seo.php (GET /seo/pages, POST /seo/pages/{rel}, POST .../suggest, `GET\	POST /seo/robots, GET\	POST /seo/sitemap), admin-v25/src/screens/LiveSeo.jsx on /seo`	The audit was a browser script, admin-seo38.js fetched all 150 pages six at a time, parsed them with DOMParser and posted its numbers back into _private/seo.json, so a stored score could say a page was an 82 while the page had no description. It is one server-side read now, and no score is stored: the route that used to receive one recomputes it. The rules became two functions with no I/O (wx_seo_issues(), wx_seo_score() - the findings and nothing else, error 8, warn 3, note 1, floored at zero), which is the only reason they are tested at all here. The page's own head stays page_meta_save's business; this screen reads it and says what Google will do with it. A schema choice is written as one managed JSON-LD block through the versioned writer, so it is in the page's history and rolls back from there. Both public files keep v2.1's refusals (Disallow: / needs asking twice; a rebuild never deletes an address no page explains), and the sitemap now follows the status out as well as in. One router change was needed: {rel:any} - a page in a folder is one parameter, not three segments nobody matches

Still to come in this phase: the integrations. The builder's 19 actions are all accounted for: 17 bridged, 2 named above as decisions.

Phase 4 - Automation and intelligence - 3 weeks

AI centre - knowledge base - AI report - SEO agent (the AI half of findability; the manager's own screen is delivered) - speed - site health - the WhatsApp suite (hub, insights, broadcasts, offers, connection) - Telegram - social planner.

Gate: an AI reply, a WhatsApp campaign and a Telegram alert are all configured, run and observed from v2.5, with the unanswered-question report showing the result.

Phase 5 - System and operations - 2 weeks

Backups (run, list, restore, download) - database browser and export - file manager - maintenance mode - system check - MCP endpoint surface - monitoring page (cron freshness, error rate, slow queries) - release artifact builder with a manifest.

Gate: a restore from backup is performed on a staging copy and the artifact manifest matches the QA run.

Phase 6 - v2.5 differentiators - 4 weeks

Feature	Why it is worth building
Real-time collaboration - SSE for chat, typing, approvals	Removes the 60-second poll that makes two agents fight over one chat
AI scope generator	Parses a brief or floor plan into a draft BOQ; the estimator rate book already exists
Client portal - tokenised, no login	Clients approve BOQs, sign and see renders; the HMAC pattern already exists in quote-view.php
Payment gateways - Stripe, PayFast, JazzCash, EasyPaisa	Manual payment entry is the biggest reconciliation cost in the invoices tracker
Inventory and procurement	Timber, acrylic, hardware and fabric tracking with purchase orders on milestone thresholds
Shared mailbox	Ends the "who replied to this client" question

Gate: a client, without an account, opens a portal link, approves a quotation and pays a deposit; the invoice reconciles itself; stock is drawn down against the project.

12. Schedule and effort

Assumes **2 developers** (one front end, one PHP/back end) plus the owner for decisions and credential rotation.

Phase	Effort (person-weeks)	Calendar (2 devs)	Cumulative
P0 Foundation OK	1	week 1	week 1
P1 Platform	3	weeks 2-3	week 3
P2 Core operations	5	weeks 4-6	week 6
P3 Content and delivery	6	weeks 7-10	week 10
P4 Automation	4	weeks 11-13	week 13
P5 System	3	weeks 14-15	week 15
P6 v2.5 differentiators	6	weeks 16-19	week 19
Total	28		

Critical path: P1 auth and store.php -> P2 core operations -> P3 pages. Any slip in P1 delays everything; P4 and P5 can run in parallel with P6 if a third developer joins.

With one developer: ~ 7 months. Recommendation: do not start P6 before P5; ship P0-P2 (6 weeks) first - that alone replaces the entire daily-operations surface.

12.1 Cost of not doing it

Cost	Evidence
Support time lost to "which build is live?"	Four admin generations, three near-identical v2.1 trees
Data-loss risk	Non-atomic JSON writes over 28 stores
Security exposure	8 Critical findings, public repository, published credentials
Feature velocity	No build step; 67 unversioned script tags; last-script-wins routing

13. Testing and QA strategy

Layer	Tool	Scope	Runs
Types	tsc --noEmit	every TS module	every push
Render smoke	src/smoke.tsx (built)	every route x every role	every push
Token contract	tests/tokens.test.ts	fails if a component contains a hex colour or a non-semantic var()	every push
API contract	tests/api/*.http	each endpoint: 200 happy path, 401 unauthenticated, 403 wrong role, 422 invalid	every push
Migration	tests/db/migrate.sh	apply to a restored copy, compare row counts and checksums, then dry-run	before each release
Money	tests/money.test.php	quotation and invoice totals against 30 recorded cases	every push
Accessibility	axe via a headless run	the 8 most-used screens	weekly
Performance	bundle budget	app <= 150 kB gz, charts lazy-loaded	every push

Bundle budget. The prototype measured 80 kB app + 160 kB vendor + 388 kB charts (uncompressed). Charts load only on screens that need them, which is why the chunk exists.

Regression rule. Every defect found after P1 gets a failing test before it gets a fix. The v2.1 audit produced 60 unique defect IDs; ten of them become permanent tests in P1 and P2.

14. Deployment and rollback

14.1 Target topology (unchanged hosting)

```
public_html/
index.html - 404.html - 500.html - 503.html - robots.txt - sitemap.xml
about/ contact/ insights/ projects/ ... 150 public pages (unchanged)
admin/ <- React build output: index.html + assets/*.[hash].js|css
api/v2/ <- PHP front controller + handlers
assets/ uploads/ backups/
_private/ <- denied by .htaccess AND by a PHP guard
.htaccess
```

.htaccess additions for the SPA: history fallback inside /admin/ only, so a deep link such as /admin/leads serves admin/index.html while the API and the public site are unaffected.

14.2 Cut-over strategy - strangler

Step	Action	Rollback
Step	Action	Rollback
1	Deploy api/v2 alongside api/. Both write the same MySQL.	Delete the folder
2	Deploy the React admin at /admin-next/, reachable by staff only	Delete the folder
3	Enable screens one group at a time via wx_settings.admin_v25_routes	Flip the setting
4	When coverage is complete, point /admin/ at the new build (302)	Restore the old admin/
5	After 14 stable days, remove frontend-v1/admin/ from the release	Restore from the artifact

At every step the v2.1 admin remains functional against the same data.

14.3 Release artifact

One zip per release with a manifest: git SHA, build hash, asset hashes, migration list applied, QA result summary, and the rollback commit. The v2.1 audit's packaging gate failed because QA results did not match a packaged artifact; the manifest is the fix.

The first such artifact exists. node tools/make-release.mjs builds woodex-master-P25/ - the live 150-page site untouched, plus api/v2/, db/ and the built console at admin-next/ - refuses to ship a credential or to build on a broken console, appends the one .htaccess block the console's deep links need, and writes MANIFEST.json: the git SHA, the console's own build stamp (the same string the bundle shows, passed to the build so the two cannot disagree), file counts and sizes, the SHA-256 of the zip and of api/v2/index.php, the migration list, and a hash per pack document. It is published as release p25 and committed in the repository; a public Drive copy of the same bytes is linked from the pack's README. Rebuilding one commit differs by exactly one line - the build stamp, which carries the minute it was built - so the artifact is bound to *commit + stamp*, and both are recorded rather than assumed.

14.4 Getting in, and knowing which build you are looking at

Two small things that between them remove the two questions asked most often about a deployment, both of which were previously answered by guessing.

The account. v2.5 signs in against wx_users - the studio's own table, filled by v2.1's first run. There is no self-service route back in if that password is lost, and on a host that has never run v2.1 there is no way to make the first account at all, because making one is behind a sign-in screen. db/owner.php is that way in, and it is deliberately boring:

```
php db/owner.php --status who can sign in, and how - no secrets, ever
php db/owner.php --email owner@woodex.com.pk create it as owner, print a password once
php db/owner.php --email admin --reset new password for an existing account
php db/owner.php --email you@x.pk --disable stop an account signing in
php db/owner.php --dry-run with any of the above, change nothing
```

It cannot read a password back (they are bcrypt hashes), it refuses a role the capability matrix does not know rather than quietly creating one, it stores names lower-case because that is how sign-in looks them up, and it prints a generated password **once**. The sign-in field takes an address *or* a bare name - v2.1's own first run writes admin, the live database has it, and an input[type=email] refuses to submit it; the server looks the name up as written, so the form does not get to decide what a sign-in name may look like. A password change bumps pw_ver, which ends every session minted under the old one, and a reset also clears a login lockout - usually the real reason the command is being run. The address, password and role rules are asserted in tests/run.php; the connection half needs a database and is exercised on the host.

The build. The strip says whether the API is answering; the console also says *which console it is*. vite.config.ts stamps the commit into the bundle - c67d564+ - built 2026-10-14 09:12Z, with a + when the working tree is dirty, because "is this the new build?" is usually asked about edits that are not committed yet,

and a clean commit id over edited files would be worse than no stamp. It shows in the sidebar foot and on the sign-in card, and a smoke assertion requires that a non-empty stamp arrived: an empty one is the failure this exists for. (The stamp replaced a fixed name in the sidebar foot - "Imtiaz Ahmad" in every state, including nobody signed in. The foot now names the person the server says is signed in, or says that nobody is.)

15. Risk register

#	Risk	Likelihood	Impact	Mitigation
R1	Credentials leak is exploited before rotation	Medium	Severe	Rotation is step 1 of P1, owner-owned, before any code deploys
R2	Rewrite drifts from the 365 existing actions	Medium	High	The alias table is generated from the live dispatcher, not written by hand
R3	Team keeps using the old admin during the transition	High	Medium	Old admin shows a banner once a screen has a v2.5 equivalent
R4	Migration corrupts documents during backfill	Low	Severe	Dry-run + checksums + restore-tested on a copy; JSON kept as payload
R5	Bundle grows past budget	Medium	Low	Budget enforced in CI; charts lazy-loaded
R6	Hostinger .htaccess quirks break the SPA fallback	Medium	Medium	Fallback scoped to /admin/; tested on a staging subdomain first
R7	Scope creep from the 12 open audit items	High	Medium	P6 is the only phase that takes new scope; P1-P5 are frozen at the gate
R8	Single developer becomes the bottleneck	Medium	High	P1 and P2 have no front-end-only work by design; the registry is data, not code

16. What the prototype already proves

Everything in admin-v25/ runs today:

Evidence	Result
Type check	clean, strict, noUnusedLocals
Render smoke	143 checks - 65 routes - 0 failures
Production build	3 chunks, content-hashed, sourcemaps on
Token contract	switching theme, accent and density re-themes all charts and screens
Routing defects	AD-3, AD-4 and AD-12 structurally impossible in the registry
Money honesty	the quotations screen labels the browser total a projection and names the endpoint
Accessibility	kanban cards move with <- / ->, not only by drag; aria-current on navigation; visible focus rings

Run it:

```
cd admin-v25 && npm install && npm run dev # http://localhost:5173
npm run verify # typecheck + 65-route smoke + build
```

17. Decisions still needed from you

#	Decision	Options	Recommendation

#	Decision	Options	Recommendation
1	Invoice print designs	one design (today) - match the three quotation designs	Build three; invoices are the document clients keep
2	Section library	51 blocks - build the 19 missing to reach the 70-block suite	Build them in P3 - they are the fastest content wins
3	Follow-the-system theme	last explicit choice - follow the OS	Last explicit choice, with a reset button
4	Repository visibility	public - private	Private - this is a Critical finding
5	Payment gateway	Stripe - PayFast - JazzCash - EasyPaisa	Start with PayFast (cards + local) in P6
6	Client portal scope	view + approve - add uploads and payments	Approve + payments first; uploads in P6.5

18. Conclusion

The fastest way to a master dashboard that the studio can rely on is **not** to rewrite the business logic that already runs a real business, and **not** to keep patching an interface whose routing model cannot be repaired. It is to replace the interface, version the API, migrate the side-stores into the database, and ship one screen group at a time behind a flag.

Six weeks of work gets the entire daily-operations surface - leads, pipeline, quotations, invoices, chat, approvals - onto the new admin with the Critical findings closed. Nine more weeks add content, automation, system control and the v2.5 features that make the product worth paying for.

Everything measured here is in docs/PROJECT-ANALYSIS-REPORT-2026-10-13.pdf; everything designed here is runnable in admin-v25/.

Appendix A - delivery gate checklist

Every phase must satisfy all six before the next begins.

- [ ] **Scope** - every screen in the phase renders for every permitted role
- [ ] **Empty** - every list has a designed empty state; every form has a designed error state
- [ ] **Data** - no screen reads a JSON side-store that migration 010 has retired
- [ ] **Money** - any figure that reaches a client document is server-computed
- [ ] **Access** - no literal colour; contrast >= 4.5:1 on text; keyboard reachable
- [ ] **Evidence** - npm run verify green, screenshots or test output attached to the phase report

Appendix B - semantic token list (33)

```
surfaces --bg --surface --surface-2 --surface-3 --surface-inset --sidebar --sidebar-hover
--sidebar-active
lines --border --border-strong
text --text --text-muted --text-subtle --text-on-accent --text-on-sidebar
--text-on-sidebar-muted
accent --accent --accent-hover --accent-soft --accent-ink --accent-ring
status --ok --ok-soft --warn --warn-soft --danger --danger-soft --info --info-soft
--neutral-soft
brand --brand --brand-soft
charts --viz-1 ... --viz-6 --grid
elevation --shadow-sm --shadow-md --shadow-lg --overlay
shape --r-xs --r-sm --r-md --r-lg --r-xl --r-full
space --s-1 --s-2 --s-3 --s-4 --s-5 --s-6 --s-8 --s-10 --s-12
type --font --font-mono --fs-11 ... --fs-28 --lh-tight --lh-snug --lh-normal
motion --dur-1 --dur-2 --dur-3 --ease
layout --sidebar-w --sidebar-w-mini --topbar-h --row-h --s-page --control-h --fs-body
depth --z-base --z-sticky --z-drawer --z-modal --z-palette --z-toast
```

Appendix C - commands

```
# prototype
cd admin-v25
npm install
npm run dev # http://localhost:5173 (binds 0.0.0.0)
npm run verify # typecheck + 65-route render smoke + production build
npm run smoke # render smoke only

# regenerate this document's PDF
python3 tools/report2pdf.py ADMIN-V2.5-MASTER-PLAN-2026-10-13.md \
ADMIN-V2.5-MASTER-PLAN-2026-10-13.pdf "Admin v2.5 Pro" \
"Master Dashboard - React + TypeScript - Phased Delivery" \
--meta "Strategy=Rewrite the front end - version the back end - preserve the data" \
--meta "Prototype=admin-v25 - 65 routes, 70 render checks, 0 failures" \
--footer "Woodex Admin v2.5 Pro Master Plan - 13 Oct 2026 - Confidential"
```

End of plan - 13 October 2026.